DEVOPS FINAL CAPSTONE · SESSION 21

SpendWise

From one commit to a monitored Kubernetes cluster

A student expense tracker I built myself and took through the whole DevOps life cycle.

APP
FastAPI · React · PostgreSQL

CI/CD
GitHub Actions · Trivy · GHCR

INFRA
Terraform · AWS VPC + EKS

DEPLOY
Kubernetes · Helm · Argo CD

OBSERVE
Prometheus · Grafana

Abhinav Suri · 24BCS10282 · SST DevOps & Cloud · github.com/Abhinavsuri90/DevOps-Capstone

THE JOURNEY

The path of one change

01
Application
FastAPI · React

02
Git commit
GitHub, main branch

03
CI pipeline
GitHub Actions, 12 jobs

04
Docker images
Backend + frontend

05
Security scan
Trivy + 5 more gates

06
Registry
GHCR, commit-SHA tags

07
Terraform
AWS VPC + EKS

08
Kubernetes
Namespace spendwise

09
Helm
One chart, three values

10
Ingress
/ and /api

11
Autoscaling
HPA, 2 to 5 Pods

12
Monitoring
Prometheus · Grafana

Then **troubleshooting**: five faults injected on purpose, found and fixed.

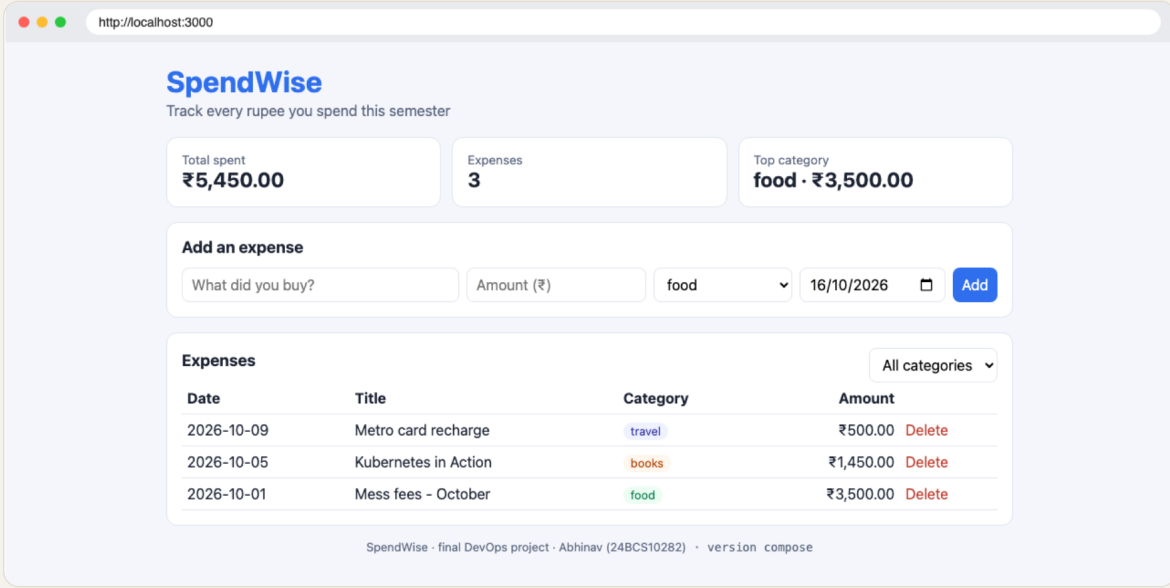
M1 · APPLICATION

A real app, written for this project

Students add what they spend and see totals by category.

Endpoint	What it does
GET /health	Liveness + database check
GET, POST /api/expenses	List with filter, create
GET, PUT, DELETE /api/expenses/{id}	Read, update, delete
GET /api/summary	Totals by category
GET /metrics	Prometheus metrics

FastAPI · SQLAlchemy · Alembic · PostgreSQL 17 · React + Vite



The app from Docker Compose at localhost:3000

M2 · TESTING

Tests are the first gate

9

pytest tests, all passing

97%

code coverage of the backend

0

real databases touched: SQLite in memory

pytest runs first in the pipeline. Every later job depends on it, so a failing test means no image is built, scanned or pushed.

`backend/tests/conftest.py · backend/pytest.ini · ruff lint`

M3 · GIT AND GITHUB

Every change is a commit

36

commits with meaningful messages

11

of them by the GitOps bot, one per release

0

secrets in the whole history (Gitleaks)

```
git log --oneline

72a2d87 gitops: deploy spendwise 28ab4ffa0951

f3fcfd3 Images and Argo CD point at DevOps-Capstone

32ce035 Final project: VPC + EKS applied, verified with kubectl
and destroyed on AWS

0cb08dd Final project: folder layout of the course brief

f9bfb4d SpendWise: new subtitle (live GitOps demo)

.gitignore: .env · __pycache__/ · node_modules/ · .venv/
· *.tfstate
```

M4 · DOCKER

Two small images, one command

backend/Dockerfile

Backend

python:3.13-alpine with pinned requirements

pip removed after install

Runs as non-root uid 10001

Migrates, then starts Uvicorn

frontend/Dockerfile

Frontend

Stage 1: node:24-alpine runs npm ci and the build

Stage 2: nginx-unprivileged serves dist/

Runs as non-root uid 101

docker-compose.yml

Compose

postgres 17, backend, frontend

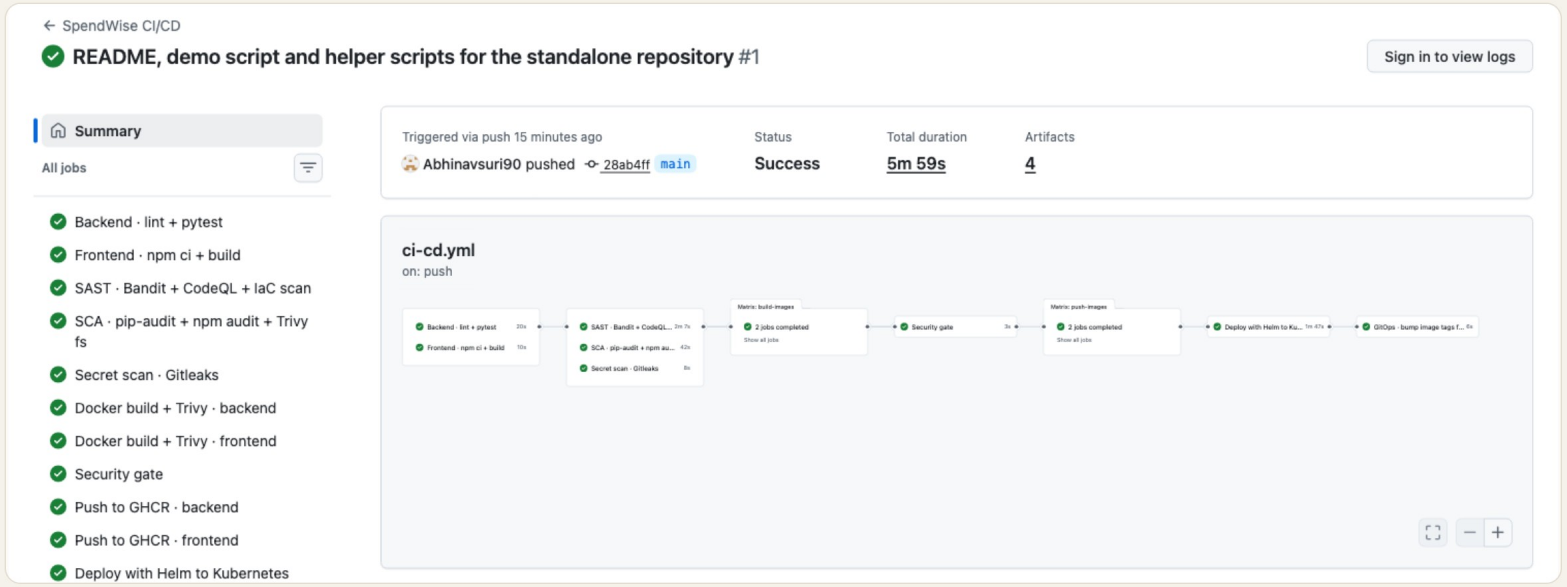
Backend waits for a healthy database

Password from a git-ignored .env

```
$ docker compose up --build  → the whole stack on localhost:3000
```

M5 · CI/CD

12 jobs on every push to main



Run 38867628237 on GitHub Actions: every job green

pytest first

A failing test stops the whole pipeline

5 min 59 s

From push to a deployed, tested release

kind + Helm

A throwaway cluster runs helm test and an Ingress smoke test

M6 · DEVSECOPS

Six gates before anything ships

Layer	Tool	Fails the pipeline on
Source code (SAST)	Bandit, CodeQL	Medium or high issues
Dependencies	pip-audit, npm audit, Trivy fs	Known vulnerabilities
Secrets	Gitleaks	Any committed secret
Both images	Trivy image, exit code 1	Fixable HIGH or CRITICAL
Infrastructure	Trivy config: Terraform + Helm	HIGH or CRITICAL misconfig
Security gate	All four results must be success	Anything else: no push

0 CRITICAL · 0 HIGH in both images

Trivy scanned the Alpine packages and every Python and npm package inside each image. A clean result means nothing at the severity we block on has a fix we could upgrade to. The IaC scan did catch real problems: Kubernetes Secrets without KMS encryption and an EKS API open to the internet. Both are fixed.

M5 · REGISTRY

Every image names its commit

COMMIT
28ab4ff

→

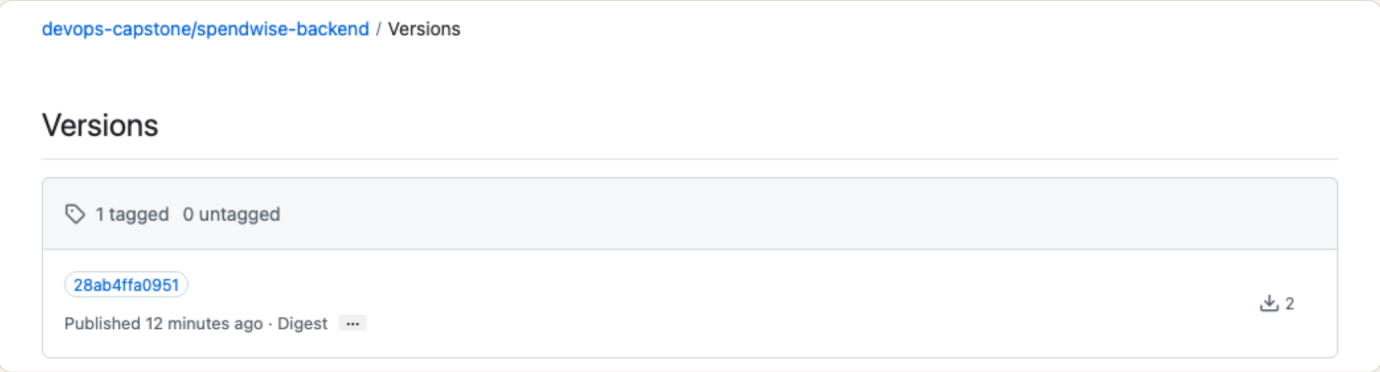
IMAGES IN GHCR
:28ab4ffa0951

→

GITOPS COMMIT
tag: 28ab4ffa0951

→

RUNNING PODS
same tag, same bytes

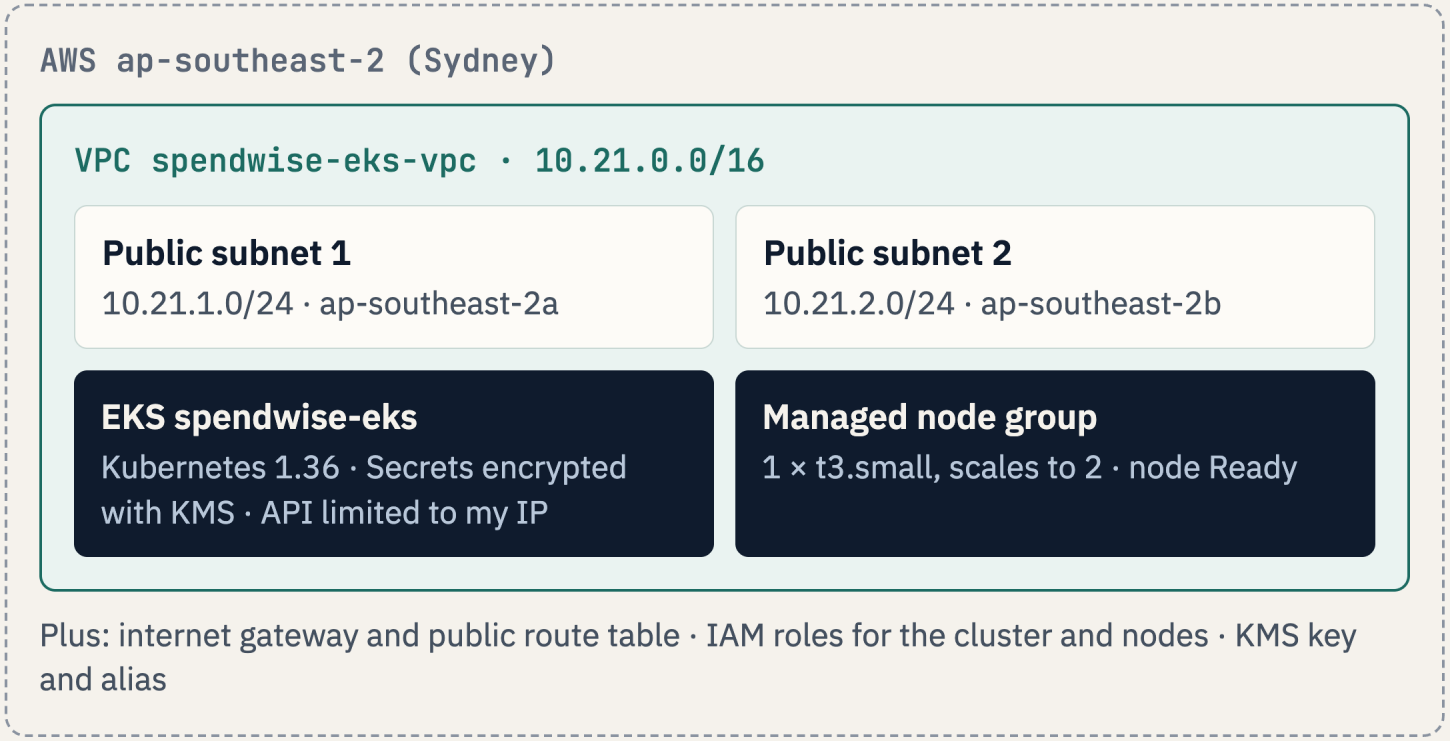


Never **latest**: the tag is the first 12 characters of the commit SHA.

The image that passed the scans is the one that runs.

M7 · TERRAFORM

AWS infrastructure as code



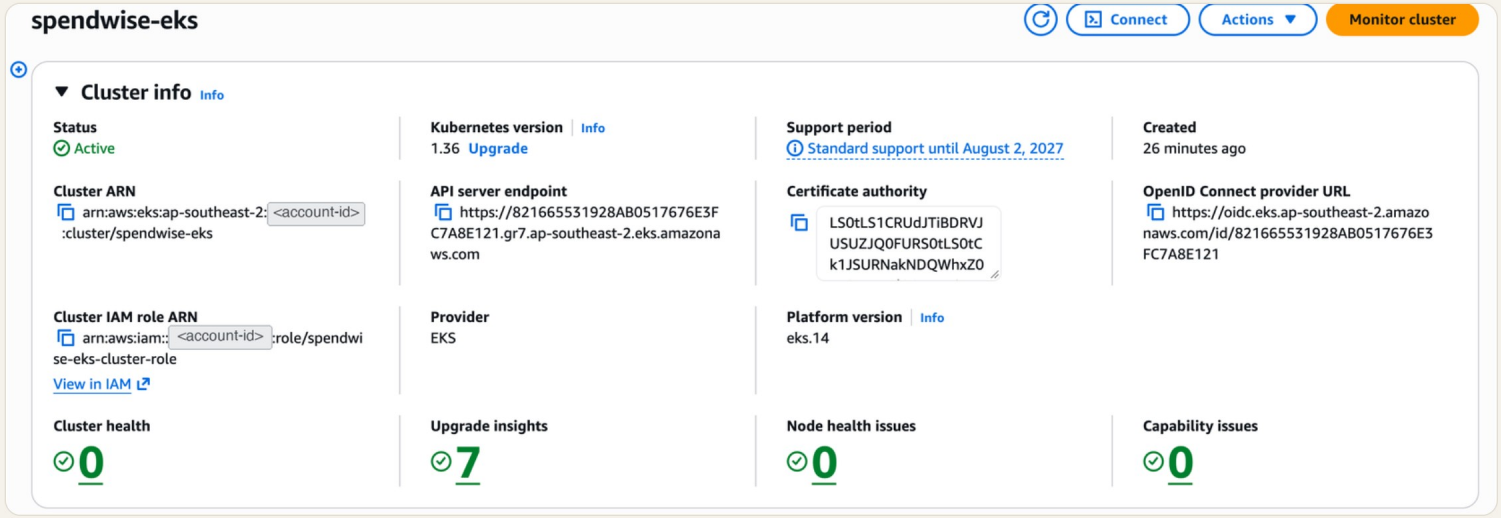
18
resources in the plan, all created

12 min
to apply; 10 of them for the EKS control plane

18
destroyed afterwards; nothing left running

M7 · AWS CONSOLE

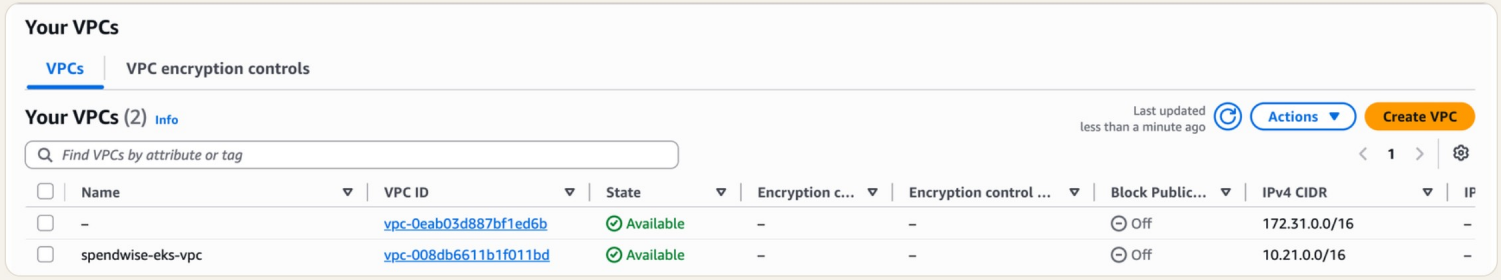
Checked in the AWS Console



EKS › Clusters › spendwise-eks

Active · Kubernetes 1.36 · 0 health issues

Compute tab: node group spendwise-eks-nodes Active



VPC › Your VPCs

spendwise-eks-vpc · 10.21.0.0/16 · Available

M8 · KUBERNETES + HELM

One Helm chart runs the stack

Object	Name	What it does
Namespace	spendwise	k8s/namespace.yaml, applied first
Deployment	spendwise-backend	FastAPI, 2–5 Pods, probes, waits for the database
Deployment	spendwise-frontend	nginx, 2 Pods, read-only root filesystem
StatefulSet	spendwise-postgres	PostgreSQL 17 with a 1 GiB PVC that keeps the data
Services	backend, frontend	ClusterIP; postgres is headless
Ingress	spendwise.local	/ → frontend, /api → backend
Secret, ConfigMap	spendwise-db, -config	Password made at deploy time, never in Git
HPA, PDB	backend, frontend	Scale at 70% CPU; keep 1 Pod during node drains

```
$ helm upgrade --install spendwise helm/spendwise -n spendwise --wait  STATUS: deployed · helm test passed
```

M8 · INGRESS + AUTOSCALING

Ingress routes, the HPA scales

BROWSER

http://spendwise.local

↓

INGRESS-NGINX

Ingress spendwise: two path rules

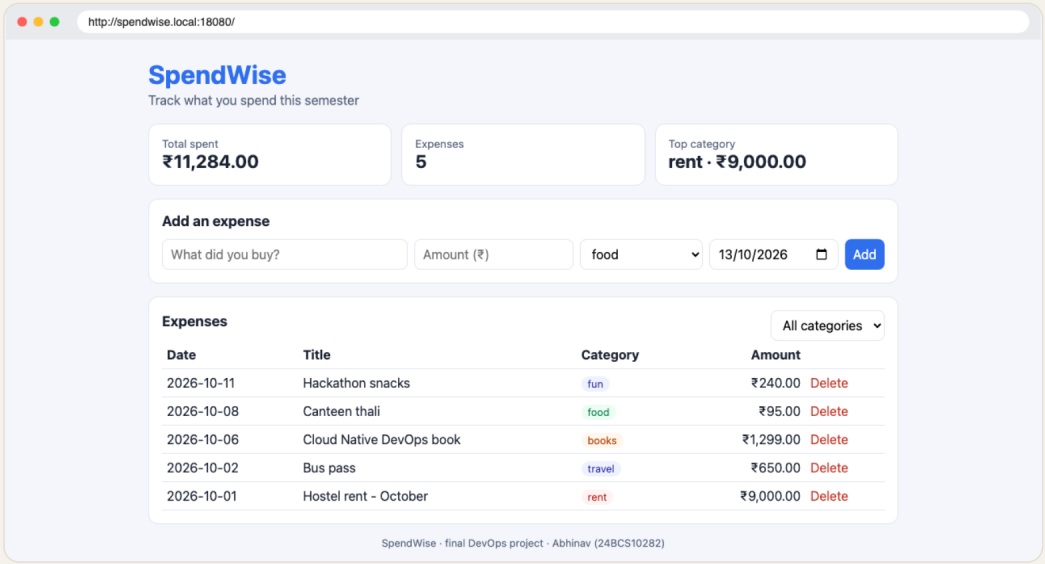
↓

/

frontend Service → 2 nginx Pods

/api

backend Service → 2–5 FastAPI Pods → PostgreSQL



HPA: 2 to 5 backend Pods at 70% CPU

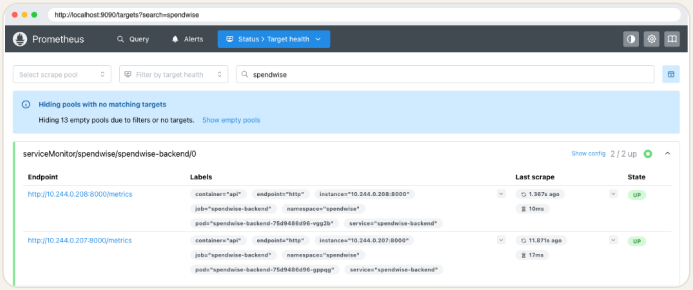
It needs CPU requests and metrics-server. Without requests it shows <unknown> and never scales: fault 5.

M9 · OBSERVABILITY

Metrics you can see live



Grafana: requests, error rate, p95 latency, expenses, Pods and HPA replicas



Prometheus: 2/2 backend targets UP

`/metrics`

Requests by route and status, a latency histogram, expenses created

`ServiceMonitor`

Shipped in the chart; Prometheus scrapes every 15 s

`Alert`

SpendWiseBackendDown fires when a target is down

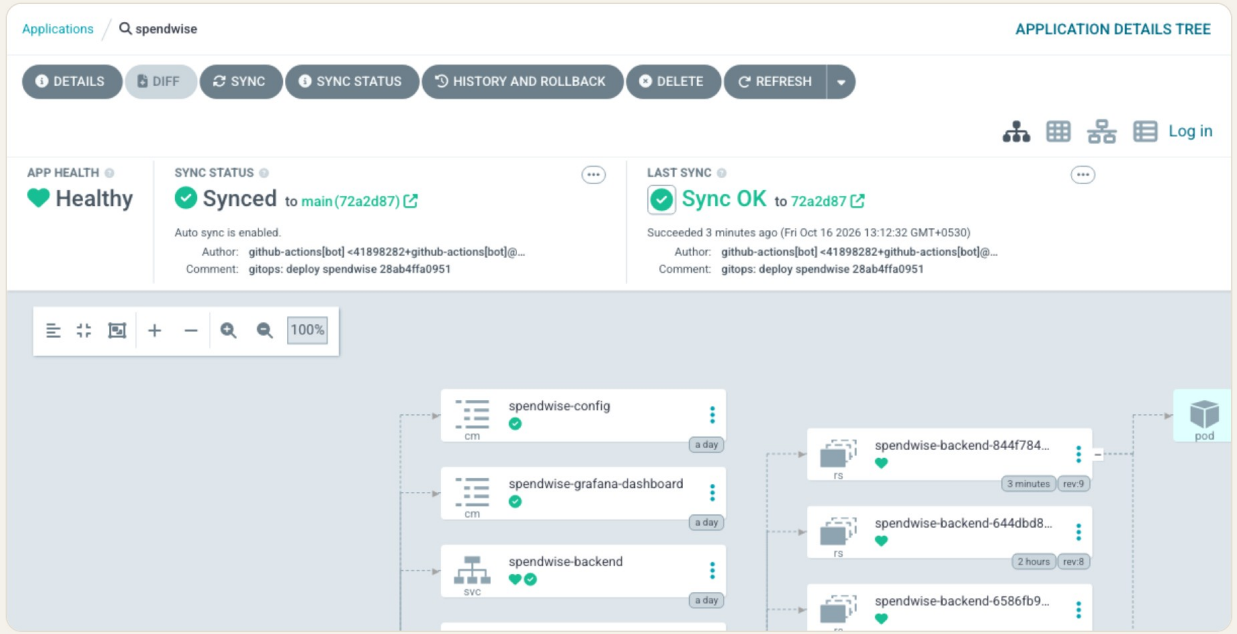
SpendWise · DevOps Capstone · Abhinav Suri · kube-prometheus-stack: Prometheus, Alertmanager, Grafana

GITOPS · ARGO CD

Git is the source of truth

1. CI pushes both images with the commit SHA as the tag
2. CI commits that tag to helm/spendwise/values-gitops.yaml
3. Argo CD polls main every 60 s and rolls out the change
4. Self-heal puts back manual changes; prune removes deleted objects

CI never holds cluster credentials.



Argo CD: Healthy, Synced to main (72a2d87) of DevOps-Capstone

TROUBLESHOOTING

Five faults, found and fixed

#	Injected fault	Symptom	Root cause	Fix
1	Image tag “locla”	ImagePullBackOff, rollout stuck	The image does not exist	helm rollback to the last good revision
2	Ingress host “spendwise.locl”	Every request returns 404	No rule matches spendwise.local	helm upgrade with the right host
3	Secret “spendwise-db-v2”	CreateContainerConfigError, API 503	The Secret does not exist	Right name, then replace the stuck Pod; data kept
4	Metrics path “/metric”	Targets DOWN, alert firing	Prometheus scrapes a 404 path	Path back to /metrics
5	CPU request removed	HPA shows cpu <unknown>	Utilisation is usage ÷ request	Requests back; scaling resumes

Same method every time: **identify → investigate → root cause → fix → verify → document.**

M10 · LIVE DEMO

One commit, nothing else by hand

git push

Change the subtitle in App.jsx and push to main

5 min 26 s

12 jobs green, new images tagged with the SHA

GitOps commit

The bot changes only the two image tags

72 s later

Argo CD Synced + Healthy; the footer shows the new version

Timings from the recorded rehearsal. Pods roll over one by one, so the app never goes down.

SUMMARY

Every module, end to end

- M1 App
- M2 Tests
- M3 Git
- M4 Docker
- M5 CI/CD
- M6 Trivy
- M7 Terraform
- M8 K8s + Helm
- M9 Monitoring
- M10 Docs

Code, evidence and every screenshot:

`github.com/Abhinavsuri90/DevOps-Capstone`

Thank you. Questions?

Abhinav Suri · 24BCS10282 · SST DevOps & Cloud